

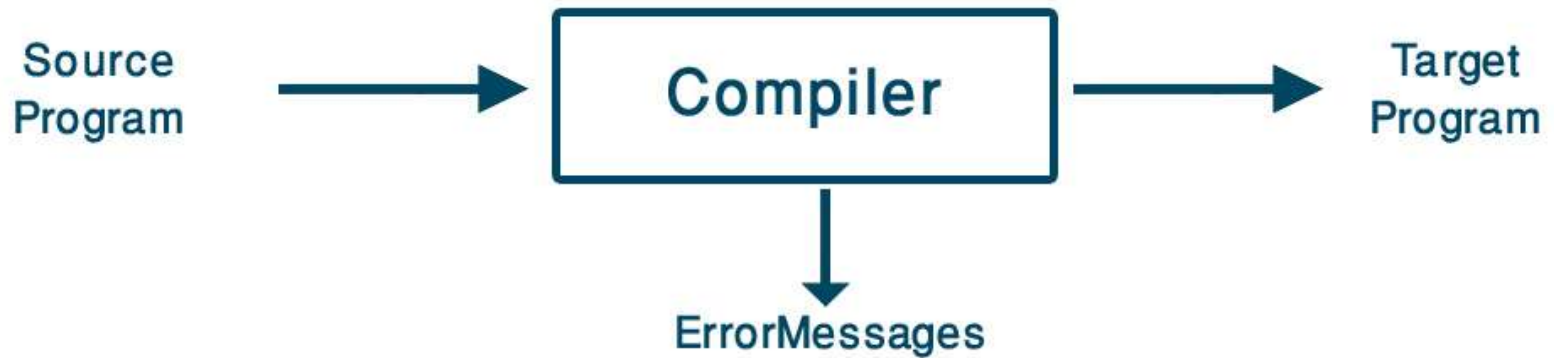
1.1 Compilers:

- A compiler is a program that reads a program written in one language — the source language — and translates it into an equivalent program in another language — the target language

1.1 Compilers:

- As an important part of this translation process, the compiler reports to its user the presence of errors in the source program.

1.1 Compilers:



1.1 Compilers:

- At first glance, the variety of compilers may appear overwhelming.
- There are thousands of source languages, ranging from traditional programming languages such as FORTRAN and Pascal to specialized languages.

1.1 Compilers:

- Target languages are equally as varied;
- A target language may be another programming language, or the machine language of any computer.

1.1 Compilers:

- Compilers are sometimes classified as:
 - single-pass
 - multi-pass
 - load-and-go
 - Debugging
 - optimizing

1.1 Compilers:

- The basic tasks that any compiler must perform are essentially the same.
- By understanding these tasks, we can construct compilers for a wide variety of source languages and target machines using the same basic techniques.

1.1 Compilers:

- Throughout the 1950's, compilers were considered notoriously difficult programs to write.
- The first FORTRAN compiler, for example, took 18 staff-years to implement.

The Analysis-Synthesis Model of Compilation:

- There are two parts of compilation:
 - Analysis
 - Synthesis

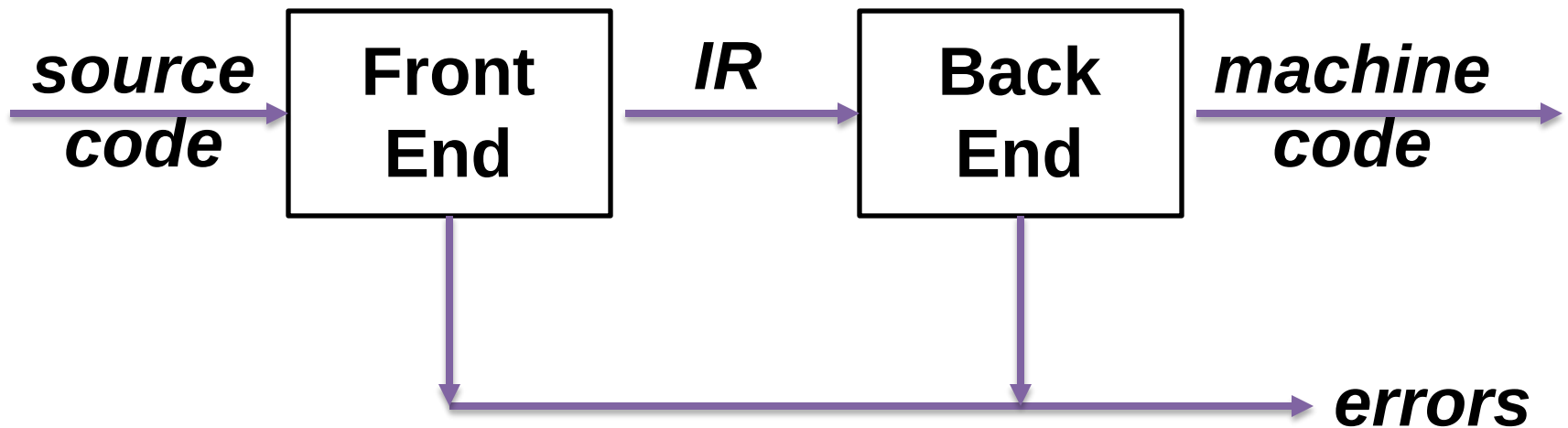
The Analysis-Synthesis Model of Compilation:

- The analysis part breaks up the source program into constituent pieces
- creates an intermediate representation of the source program.

The Analysis-Synthesis Model of Compilation:

- The synthesis part constructs the desired target program from the intermediate representation.

The Analysis-Synthesis Model of Compilation:



The Analysis-Synthesis Model of Compilation:

- During analysis, the operations implied by the source program are determined and recorded in a hierarchical structure called a tree.
- Often, a special kind of tree called a syntax tree is used.

The Analysis-Synthesis Model of Compilation:

- In syntax tree each node represents an operation and the children of the node represent the arguments of the operation.
- For example, a syntax tree of an assignment statement is shown below.

The Analysis-Synthesis Model of Compilation:

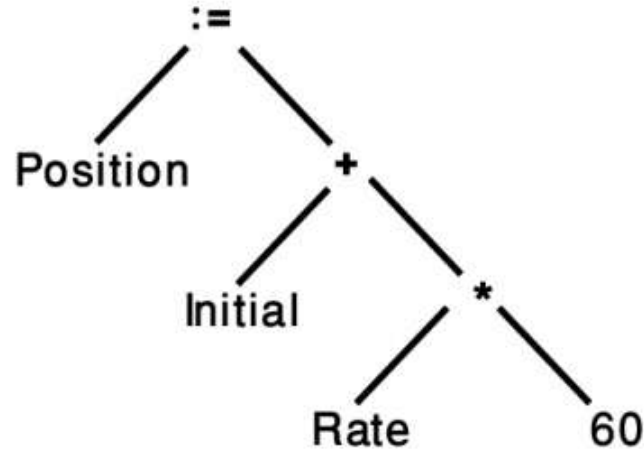


Fig 1.2. Syntax tree of position :=initial + rate * 60

Analysis of the Source Program:

- In compiling, analysis consists of three phases:
 - **Linear Analysis:**
 - **Hierarchical Analysis:**
 - **Semantic Analysis:**

Analysis of the Source Program:

- **Linear Analysis:**
 - In which the stream of characters making up the source program is read from left-to-right and grouped into tokens that are sequences of characters having a collective meaning.

Scanning or Lexical Analysis (Linear Analysis):

- In a compiler, linear analysis is called lexical analysis or scanning.
- For example, in lexical analysis the characters in the assignment statement
 - $\text{Position:} = \text{initial} + \text{rate} * 60$
- Would be grouped into the following tokens:

Scanning or Lexical Analysis (Linear Analysis):

- The identifier, position.
- The assignment symbol :=
- The identifier initial.
- The plus sign.
- The identifier rate.
- The multiplication sign.
- The number 60.

Scanning or Lexical Analysis (Linear Analysis):

- The blanks separating the characters of these tokens would normally be eliminated during the lexical analysis.

Analysis of the Source Program:

- **Hierarchical Analysis:**
 - In which characters or tokens are grouped hierarchically into nested collections with collective meaning.

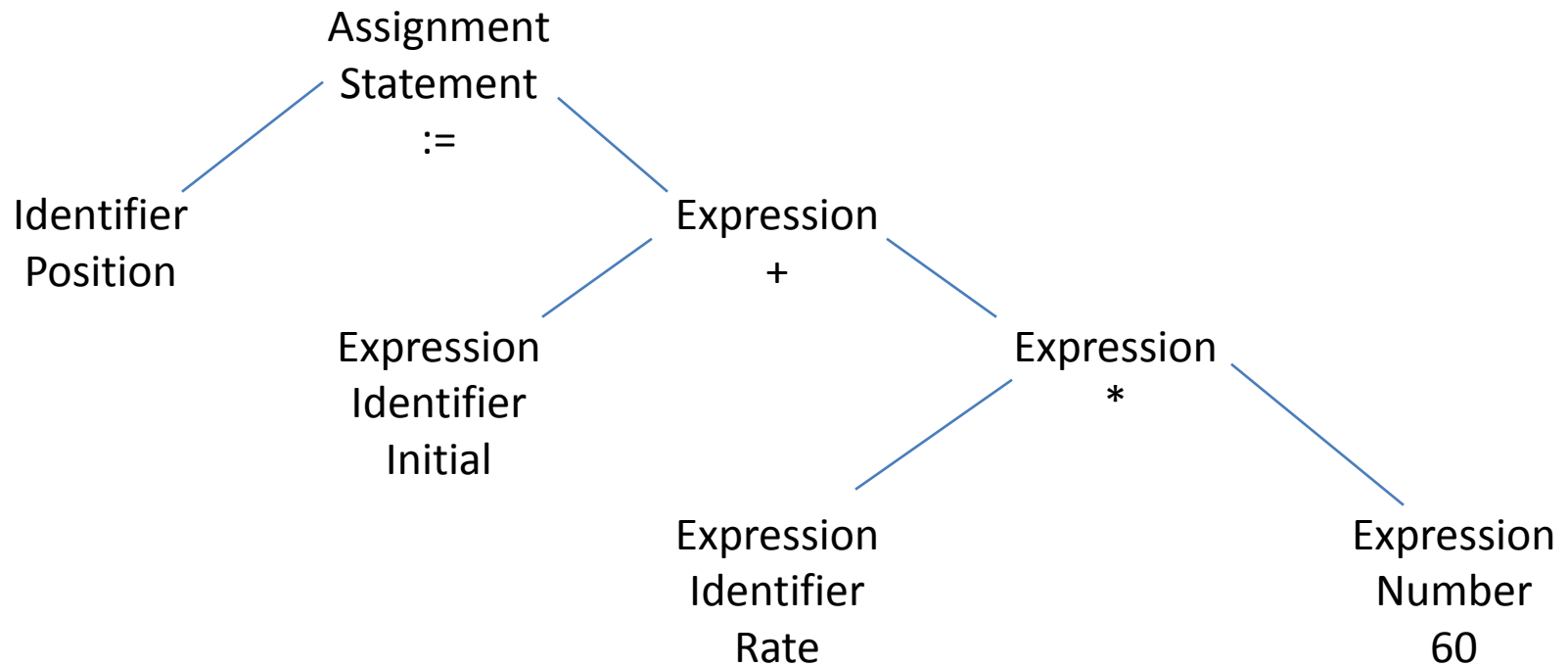
Syntax Analysis or Hierarchical Analysis (Parsing):

- Hierarchical analysis is called parsing or syntax analysis.
- It involves grouping the tokens of the source program into grammatical phases that are used by the compiler to synthesize output.

Syntax Analysis or Hierarchical Analysis (Parsing):

- The grammatical phrases of the source program are represented by a parse tree.

Syntax Analysis or Hierarchical Analysis (Parsing):



Syntax Analysis or Hierarchical Analysis (Parsing):

- In the expression **initial + rate * 60**, the phrase **rate * 60** is a logical unit because the usual conventions of arithmetic expressions tell us that the multiplication is performed before addition.
- Because the expression **initial + rate** is followed by a *****, it is not grouped into a single phrase by itself

Syntax Analysis or Hierarchical Analysis (Parsing):

- The hierarchical structure of a program is usually expressed by recursive rules.
- For example, we might have the following rules, as part of the definition of expression:

Syntax Analysis or Hierarchical Analysis (Parsing):

- Any identifier is an expression.
- Any number is an expression
- If expression_1 and expression_2 are expressions, then so are
 - $\text{Expression}_1 + \text{expression}_2$
 - $\text{Expression}_1 * \text{expression}_2$
 - (Expression_1)

Analysis of the Source Program:

- **Semantic Analysis:**
 - In which certain checks are performed to ensure that the components of a program fit together meaningfully.

Semantic Analysis:

- The semantic analysis phase checks the source program for semantic errors and gathers type information for the subsequent code-generation phase.

Semantic Analysis:

- It uses the hierarchical structure determined by the syntax-analysis phase to identify the operators and operand of expressions and statements.

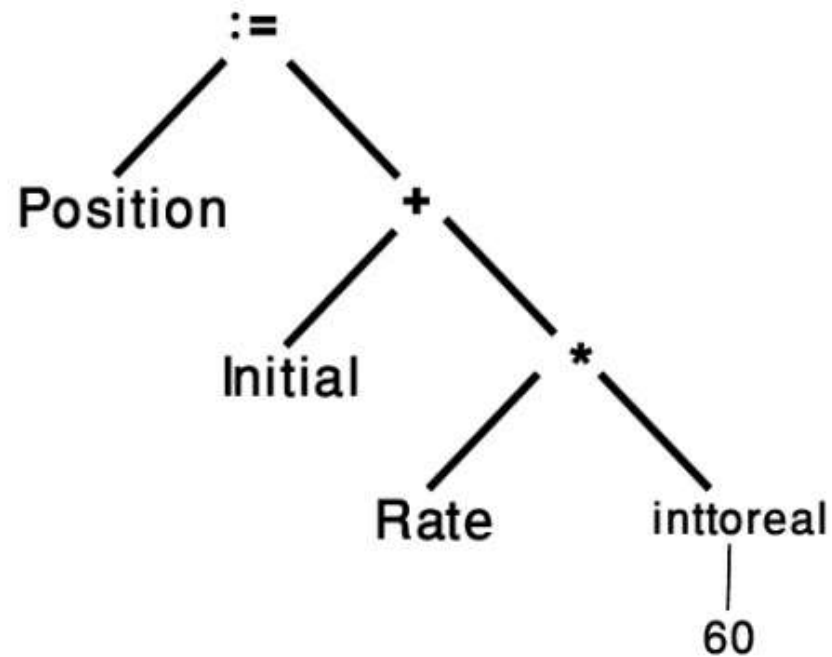
Semantic Analysis:

- An important component of semantic analysis is type checking.
- Here are the compiler checks that each operator has operands that are permitted by the source language specification.

Semantic Analysis:

- For example, when a binary arithmetic operator is applied to an integer and real. In this case, the compiler may need to be converting the integer to a real. As shown in figure given below

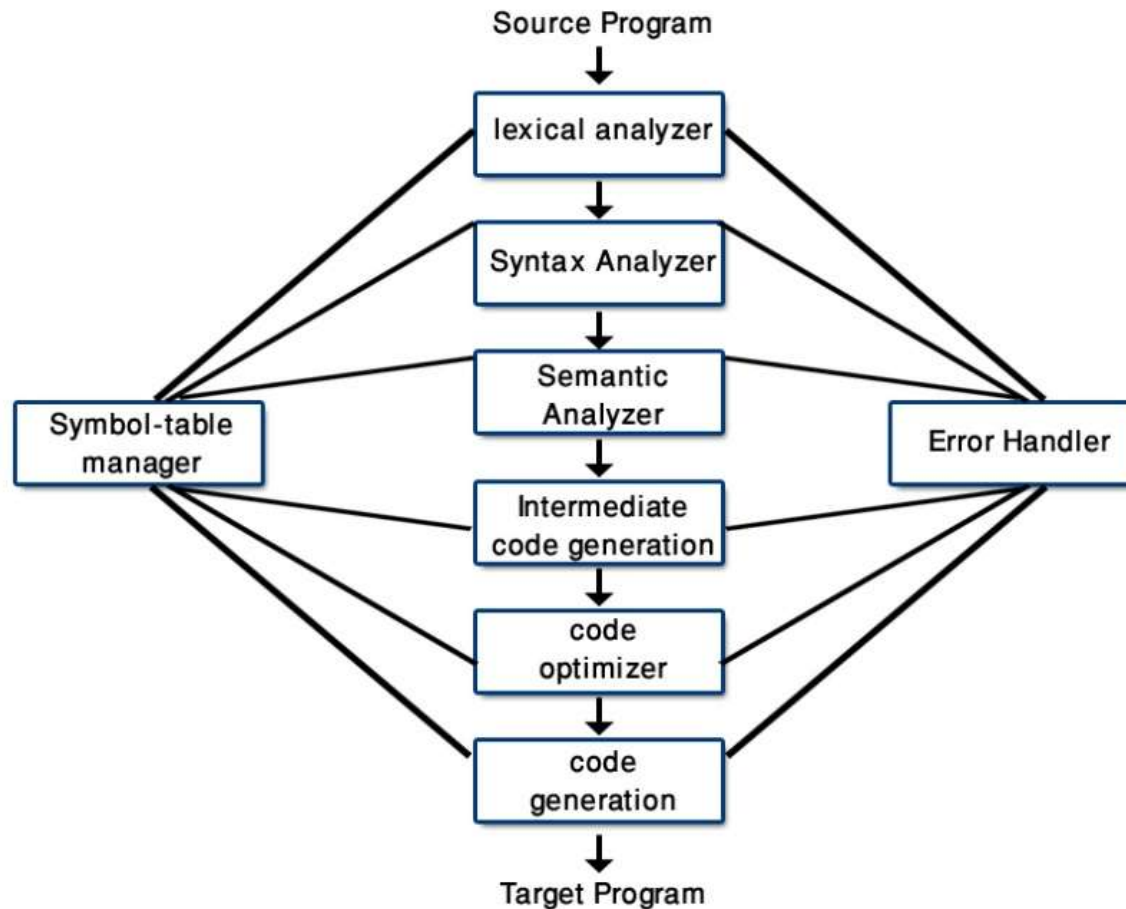
Semantic Analysis:



1.3 The Phases of a Compiler:

- A compiler operates in phases.
- Each of which transforms the source program from one representation to another.
- A typical decomposition of a compiler is shown in fig given below

1.3 The Phases of a Compiler:



1.3 The Phases of a Compiler:

- **Linear Analysis:**
 - In which the stream of characters making up the source program is read from left-to-right and grouped into tokens that are sequences of characters having a collective meaning.

1.3 The Phases of a Compiler:

- In a compiler, linear analysis is called lexical analysis or scanning.
- For example, in lexical analysis the characters in the assignment statement
 - $\text{Position} = \text{initial} + \text{rate} * 60$
- Would be grouped into the following tokens:

1.3 The Phases of a Compiler:

- The identifier, position.
- The assignment symbol :=
- The identifier initial.
- The plus sign.
- The identifier rate.
- The multiplication sign.
- The number 60.

1.3 The Phases of a Compiler:

- The blanks separating the characters of these tokens would normally be eliminated during the lexical analysis.

1.3 The Phases of a Compiler:

- **Hierarchical Analysis:**
 - In which characters or tokens are grouped hierarchically into nested collections with collective meaning.

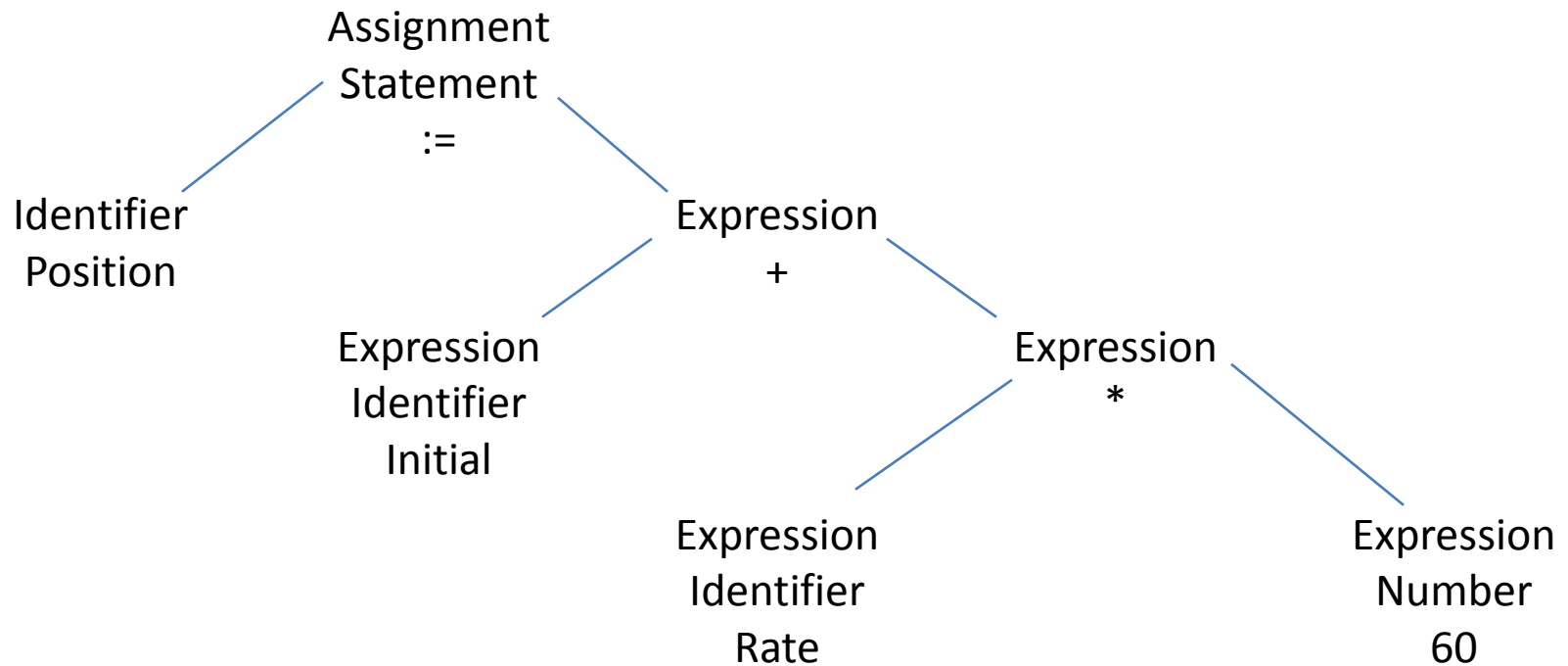
1.3 The Phases of a Compiler:

- Hierarchical analysis is called parsing or syntax analysis.
- It involves grouping the tokens of the source program into grammatical phases that are used by the compiler to synthesize output.

1.3 The Phases of a Compiler:

- The grammatical phrases of the source program are represented by a parse tree.

1.3 The Phases of a Compiler:



1.3 The Phases of a Compiler:

- In the expression **initial + rate * 60**, the phrase **rate * 60** is a logical unit because the usual conventions of arithmetic expressions tell us that the multiplication is performed before addition.
- Because the expression **initial + rate** is followed by a *****, it is not grouped into a single phrase by itself

1.3 The Phases of a Compiler:

- The hierarchical structure of a program is usually expressed by recursive rules.
- For example, we might have the following rules, as part of the definition of expression:

1.3 The Phases of a Compiler:

- Any identifier is an expression.
- Any number is an expression
- If expression_1 and expression_2 are expressions, then so are
 - $\text{Expression}_1 + \text{expression}_2$
 - $\text{Expression}_1 * \text{expression}_2$
 - (Expression_1)

1.3 The Phases of a Compiler:

- **Semantic Analysis:**
 - In which certain checks are performed to ensure that the components of a program fit together meaningfully.

1.3 The Phases of a Compiler:

- The semantic analysis phase checks the source program for semantic errors and gathers type information for the subsequent code-generation phase.

1.3 The Phases of a Compiler:

- It uses the hierarchical structure determined by the syntax-analysis phase to identify the operators and operand of expressions and statements.

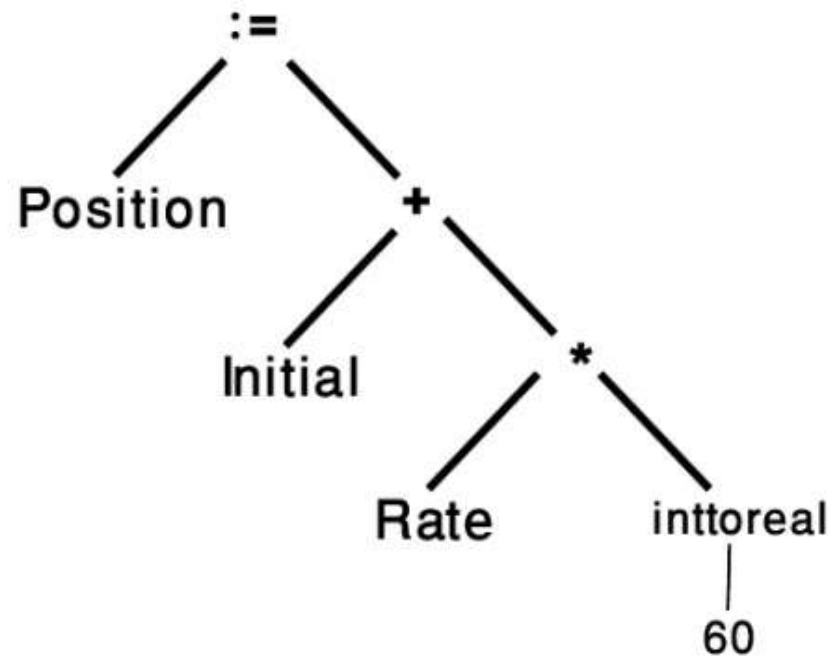
1.3 The Phases of a Compiler:

- An important component of semantic analysis is type checking.
- Here are the compiler checks that each operator has operands that are permitted by the source language specification.

1.3 The Phases of a Compiler:

- For example, when a binary arithmetic operator is applied to an integer and real. In this case, the compiler may need to be converting the integer to a real. As shown in figure given below

1.3 The Phases of a Compiler:



1.3 The Phases of a Compiler:

- **Symbol Table Management:**
 - An essential function of a compiler is to record the identifiers used in the source program and collect information about various attributes of each identifier.
 - These attributes may provide information about the storage allocated for an identifier, its type, its scope.

1.3 The Phases of a Compiler:

- The symbol table is a data structure containing a record for each identifier with fields for the attributes of the identifier.
- When an identifier in the source program is detected by the lexical analyzer, the identifier is entered into the symbol table

1.3 The Phases of a Compiler:

- However, the attributes of an identifier cannot normally be determined during lexical analysis.
- For example, in a Pascal declaration like
- **Var position, initial, rate : real;**
- The type real is not known when position, initial and rate are seen by the lexical analyzer.

1.3 The Phases of a Compiler:

- The remaining phases gets information about identifiers into the symbol table and then use this information in various ways.
- For example, when doing semantic analysis and intermediate code generation, we need to know what the types of identifiers are, so we can check that the source program uses them in valid ways, and so that we can generate the proper operations on them.

1.3 The Phases of a Compiler:

- The code generator typically enters and uses detailed information about the storage assigned to identifiers.

Error Detection and Reporting:

- Each phase can encounter errors.
- However, after detecting an error, a phase must somehow deal with that error, so that compilation can proceed, allowing further errors in the source program to be detected.

Error Detection and Reporting:

- A compiler that stops when it finds the first error is not as helpful as it could be.
- The syntax and semantic analysis phases usually handle a large fraction of the errors detectable by the compiler.

Error Detection and Reporting:

- Errors where the token stream violates the structure rules (syntax) of the language are determined by the syntax analysis phase.
- The lexical phase can detect errors where the characters remaining in the input do not form any token of the language.

Intermediate Code Generation:

- After Syntax and semantic analysis, some compilers generate an explicit intermediate representation of the source program.
- We can think of this intermediate representation as a program for an abstract machine.

Intermediate Code Generation:

- This intermediate representation should have two important properties;
 - it should be easy to produce,
 - easy to translate into the target program.

Intermediate Code Generation:

- We consider an intermediate form called “three-address code,”
- which is like the assembly language for a machine in which every memory location can act like a register.

Intermediate Code Generation:

- Three-address code consists of a sequence of instructions, each of which has at most three operands.
- The source program in (1.1) might appear in three-address code as

Intermediate Code Generation:

(1.3)

- Temp1 := inttoreal (60)
- Temp2 := id3 * temp1
- Temp3 := id2 + temp2
- id1 := temp3

Code Optimization:

- The code optimization phase attempts to improve the intermediate code, so that faster-running machine code will result.

Code Optimization:

- Some optimizations are trivial.
- For example, a natural algorithm generates the intermediate code (1.3), using an instruction for each operator in the tree representation after semantic analysis, even though there is a better way to perform the same calculation, using the two instructions.

Code Optimization:

(1.4)

- Temp1 := id3 * 60.0
- id := id2 + temp1
- There is nothing wrong with this simple algorithm, since the problem can be fixed during the code-optimization phase.

Code Optimization:

- That is, the compiler can deduce that the conversion of 60 from integer to real representation can be done once and for all at compile time, so the `inttoreal` operation can be eliminated.

Code Optimization:

- Besides, temp3 is used only once, to transmit its value to id1. It then becomes safe to substitute id1 for temp3, whereupon the last statement of 1.3 is not needed and the code of 1.4 results.

Code Generation

- The final phase of the compiler is the generation of target code
- consisting normally of relocatable machine code or assembly code.

Code Generation

- Memory locations are selected for each of the variables used by the program.
- Then, intermediate instructions are each translated into a sequence of machine instructions that perform the same task.
- A crucial aspect is the assignment of variables to registers.

Code Generation

- For example, using registers 1 and 2, the translation of the code of 1.4 might become
- `MOVF id3, r2`
- `MULF #60.0, r2`
- `MOVF id2, r1`
- `ADDF r2, r1`
- `MOVF r1, id1`

Code Generation

- The first and second operands of each instruction specify a source and destination, respectively.
- The F in each instruction tells us that instructions deal with floating-point numbers.

Code Generation

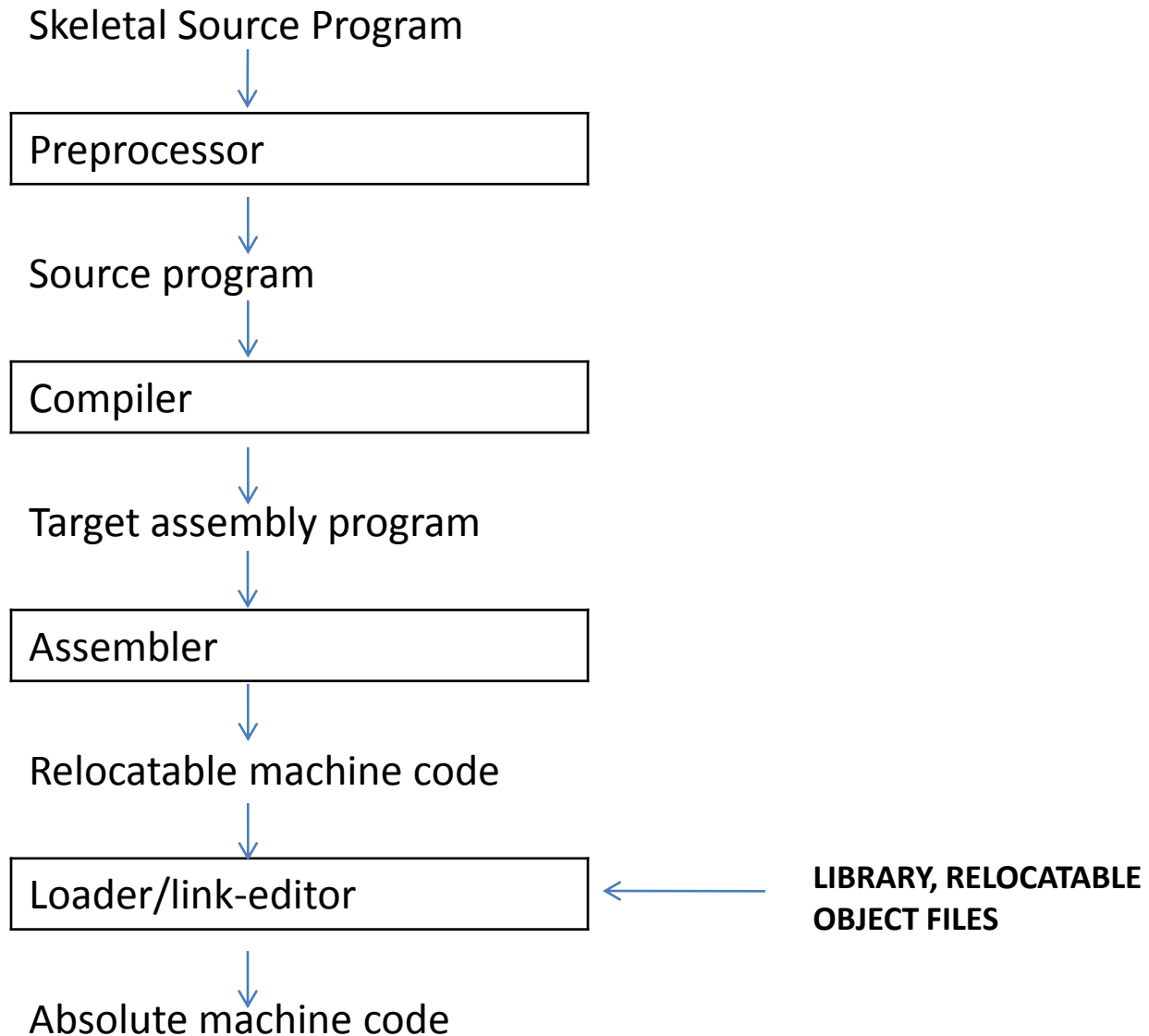
- This code moves the contents of the address id3 into register 2, and then multiplies it with the real-constant 60.0.
- The # signifies that 60.0 is to be treated as a constant.

Code Generation

- The third instruction moves id2 into register 1 and adds to it the value previously computed in register 2
- Finally, the value in register 1 is moved into the address of id1.

1.4 Cousins of the Compiler:

- As we saw in given figure, the input to a compiler may be produced by one or more preprocessors, and further processing of the compiler's output may be needed before running machine code is obtained.



1.3. A LANGUAGE-PROCESSING SYSTEM

1.4 Cousins of the Compiler:

- **Preprocessors:**
- preprocessors produce input to compilers.
They may perform the following functions:
 - **Macro Processing:**
 - **File inclusion:**
 - **“Rational” Preprocessors:**
 - **Language extensions:**

Preprocessors:

- **Macro Processing:**

- A preprocessor may allow a user to define macros that are shorthand's for longer constructs.

Preprocessors:

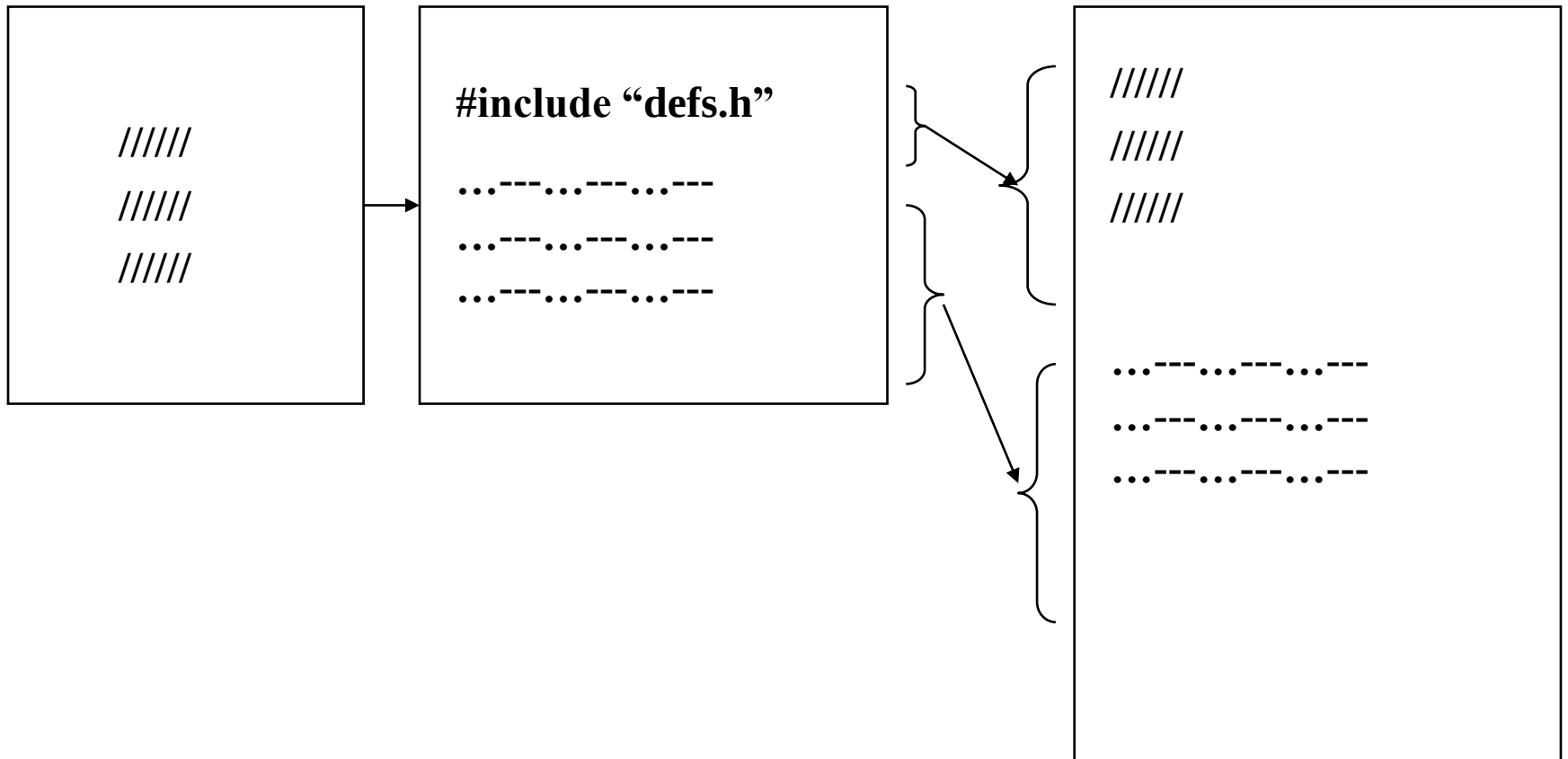
- **File inclusion:**

- A preprocessor may include header files into the program text.
- For example, the C preprocessor causes the contents of the file `<global.h>` to replace the statement `#include <global.h>` when it processes a file containing this statement.

Preprocessors:

defs.h

main.c



Preprocessors:

- **“Rational” Preprocessors:**
 - These processors augment older languages with more modern flow-of-control and data-structuring facilities.

Preprocessors:

- **Language extensions:**
 - These processors attempt to add capabilities to the language by what amounts to built-in macros.
 - For example, the language Equal is a database query language embedded in C. Statements beginning with `##` are taken by the preprocessor to be database-access statements, unrelated to C, and are translated into procedure calls on routines that perform the database access.

Assemblers:

- Some compilers produce assembly code that is passed to an assembler for further processing.
- Other compilers perform the job of the assembler, producing relocatable machine code that can be passed directly to the loader/link-editor.

Assemblers:

- Here we shall review the relationship between assembly and machine code.

Assemblers:

- Assembly code is a mnemonic version of machine code.
- In which names are used instead of binary codes for operations, and names are also given to memory addresses.

Assemblers:

- A typical sequence of assembly instructions might be
 - MOV a , R1
 - ADD #2 , R1
 - MOV R1 , b

Assemblers:

- This code moves the contents of the address `a` into register 1, then adds the constant 2 to it, reading the contents of register 1 as a fixed-point number, and finally stores the result in the location named by `b`. thus, it computes $b := a + 2$.

Two-Pass Compiler:

Two-Pass Compiler:

- The simplest form of assembler makes two passes over the input.

Two-Pass Compiler:

- in the first pass, all the identifiers that denote storage locations are found and stored in a symbol table
- Identifiers are assigned storage locations as they are encountered for the first time, so after reading 1.6, for example, the symbol table might contain the entries shown in given below.

Two-Pass Compiler:

MOV a , R1	Identifiers	Address
ADD #2 , R1	a	0
MOV R1 , b	b	4

Two-Pass Compiler:

- In the second pass, the assembler scans the input again.
- This time, it translates each operation code into the sequence of bits representing that operation in machine language.
- The output of the 2nd pass is usually relocatable machine code.

Loaders and Link-Editors:

- usually, a program called a loader performs the two functions of loading and link-editing.

Loaders and Link-Editors:

- The process of **loading** consists of taking relocatable machine code, altering the relocatable addresses, and placing the altered instructions and data in memory at the proper location.

Loaders and Link-Editors:

- The **link-editor** allows us to make a single program from several files of relocatable machine code.

1.5

The Grouping of Phases:

Front and Back Ends:

- The phases are collected into a front end and a back end.
- The front end consists of those phases that depend primarily on the source language and are largely independent of the target machine.

Front and Back Ends:

- These normally include lexical and syntactic analysis, the creating of the symbol table, semantic analysis, and the generation of intermediate code.
- A certain amount of code optimization can be done by the front end as well.

Front and Back Ends:

- The front end also includes the error handling that goes along with each of these phases.

Front and Back Ends:

- The back end includes those portions of the compiler that depend on the target machine.
- And generally, these portions do not depend on the source language, depend on just the intermediate language.

Front and Back Ends:

- In the back end, we find aspects of the code optimization phase, and we find code generation, along with the necessary error handling and symbol table operations.

Passes:

Compiler-Construction Tools:

- The compiler writer, like any programmer, can profitably use tools such as
 - Debuggers,
 - Version managers,
 - Profilers and so on.

Compiler-Construction Tools:

- In addition to these software-development tools, other more specialized tools have been developed for helping implement various phases of a compiler.

Compiler-Construction Tools:

- Shortly after the first compilers were written, systems to help with the compiler-writing process appeared.
- These systems have often been referred to as
 - Compiler-compilers,
 - Compiler-generators,
 - Or Translator-writing systems.

Compiler-Construction Tools:

- Some general tools have been created for the automatic design of specific compiler components.
- These tools use specialized languages for specifying and implementing the component, and many use algorithms that are quite sophisticated.

Compiler-Construction Tools:

- The most successful tools are those that hide the details of the generation algorithm and produce components that can be easily integrated into the remainder of a compiler.

Compiler-Construction Tools:

- The following is a list of some useful compiler-construction tools:
 - **Parser generators**
 - **Scanner generators**
 - **Syntax directed translation engines**
 - **Automatic code generators**
 - **Data-flow engines**

Compiler-Construction Tools:

- **Parser generators**

- These produce syntax analyzers, normally from input that is based on a context-free grammar.
- In early compilers, syntax analysis consumed not only a large fraction of the running time of a compiler, but a large fraction of the intellectual effort of writing a compiler.
- This phase is considered one of the easiest to implement.

Compiler-Construction Tools:

- **Scanner generators:**
 - These tools automatically generate lexical analyzers, normally from a specification based on regular expressions.
 - The basic organization of the resulting lexical analyzer is in effect a finite automaton.

Compiler-Construction Tools:

- **Syntax directed translation engines:**
 - These produce collections of routines that walk the parse tree, generating intermediate code.
 - The basic idea is that one or more “translations” are associated with each node of the parse tree, and each translation is defined in terms of translations at its neighbor nodes in the tree.

Compiler-Construction Tools:

- **Automatic code generators:**
 - Such a tool takes a collection of rules that define the translation of each operation of the intermediate language into the machine language for the target machine.

- **Data-flow engines:**

- Much of the information needed to perform good code optimization involves “data-flow analysis,” the gathering of information how values are transmitted from one part of a program to each other part.